Day 100 — Line-by-Line Syntax Explanation

MD Mutasim Billah | 52-Week Data Science to ML/AI Roadmap | Week 15, Day 3 reference

Watch: Named Entity Recognition - using BERT model (Search Techie) | Named Entity Recognition, Part 5 (Gyan DEEP)

`day100_named_entity_recognition.py`, segment by segment, in the order `main()` uses things. The model and the data come from the libraries; the alignment, span decoding, scoring and gazetteer are from scratch. Code boxes are cut from the real script by program (inline comments removed). Purple **MATH** boxes come before code that implements a formula. A glossary is at the end.

MATH BOX LEGEND

Symbols: B-/I-/O = begin / inside / outside tags · (s, e, type) = a span, end exclusive · P, R, F1 = precision, recall, F1 over exact spans

Segment 0 — Setup

Imports and settings

CODE

```
from transformers import AutoModelForTokenClassification, AutoTokenizer, pipeline
from transformers.utils import logging as hf_logging
from huggingface_hub.utils import logging as hub_logging
import matplotlib
matplotlib.use("Agg")
import matplotlib.pyplot as plt
from matplotlib.patches import FancyBboxPatch
MODEL = "dslim/distilbert-NER"
TAGS = ["O", "B-PER", "I-PER", "B-ORG", "I-ORG", "B-LOC", "I-LOC", "B-MISC", "I-MISC"]
TYPES = ["PER", "ORG", "LOC", "MISC"]
```

- `AutoModelForTokenClassification` — a Transformer with a classifier at every position: one tag per piece.
- `TAGS = [...]` — CoNLL's tag ids in order; index 3 is B-ORG, and so on.

Segment 1 — Data and alignment

conll: the data from the hub

CODE

```
def conll(split):
    d = load_dataset("eriktks/conll2003", revision="refs/convert/parquet", split=split)
    return list(d["tokens"]), [[TAGS[t] for t in tags] for tags in d["ner_tags"]]
```

- `revision="refs/convert/parquet"` — a branch of the dataset repository holding plain parquet files, readable without a loading script.
- `[[TAGS[t] for t in tags] for tags in ...]` — a nested list comprehension: tag ids to tag names, sentence by sentence.

predict_tags: pieces back to words

CODE

```
def predict_tags(tok, model, sents, bs=32):
    out = []
    with torch.no_grad():
        for i in range(0, len(sents), bs):
            batch = sents[i:i + bs]
            enc = tok(batch, is_split_into_words=True, padding=True, truncation=True,
                return_tensors="pt")
            pred = model(**enc).logits.argmax(-1).numpy()
            for b, words in enumerate(batch):
                tags, seen = [], set()
                for pos, w in enumerate(enc.word_ids(b)):
                    if w is not None and w not in seen:
                        tags.append(TAGS[pred[b, pos]]); seen.add(w)
                tags += ["O"] * (len(words) - len(tags))
                out.append(tags)
    return out
```

- `is_split_into_words=True` — the input is already a list of words; the tokenizer splits each word into pieces and remembers which is which.
- `enc.word_ids(b)` — for sentence b in the batch: the word index of every piece (None for [CLS], [SEP], padding).
- `if w is not None and w not in seen:` — only a word's first piece is used; seen is a set of word indices already tagged.

Segment 2 — Scoring

spans: BIO tags to (start, end, type)

CODE

```
def spans(tags):
    out, start, typ = set(), None, None
    for i, t in enumerate(tags + ["O"]):
        if t == "O" or t.startswith("B-") or (t.startswith("I-") and t[2:] != typ):
            if start is not None:
                out.add((start, i, typ))
            start, typ = (i, t[2:]) if t != "O" else (None, None)
    return out
```

- `tags + ["O"]` — a sentinel O at the end closes a name that runs to the last word.
- `t[2:] != typ` — an I- tag of a different type cannot continue the open name, so it closes it and opens a new one.
- `out.add((start, i, typ))` — a set of tuples: easy to intersect with the gold spans.

prf and by_type

MATH

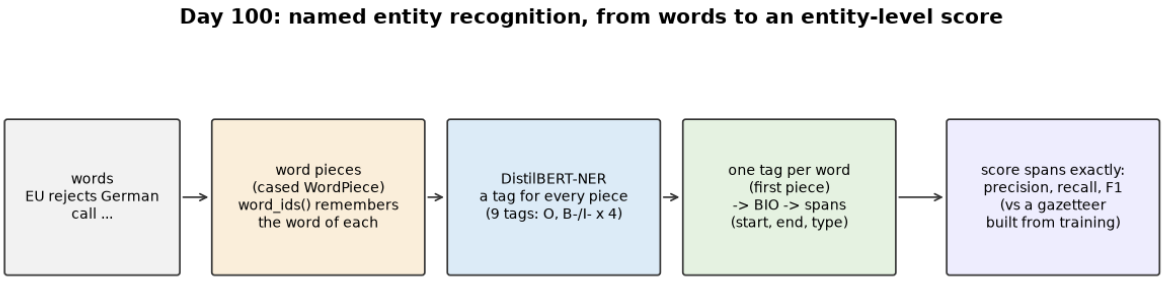
$$P = |G \cap Pr| / |Pr|, \quad R = |G \cap Pr| / |G|, \quad F1 = 2PR / (P + R)$$

CODE

```
def prf(gold, pred):
    tp = sum(len(g & p) for g, p in zip(gold, pred))
    n_pred, n_gold = sum(map(len, pred)), sum(map(len, gold))
    p, r = tp / max(1, n_pred), tp / max(1, n_gold)
    return p, r, (2 * p * r / (p + r) if p + r else 0.0), tp, n_pred, n_gold
def by_type(gold, pred, typ):
    keep = lambda S: [{s for s in x if s[2] == typ} for x in S]
    return prf(keep(gold), keep(pred))
```

- `len(g & p)` — & on two sets is their intersection: spans that match exactly.
- `sum(map(len, pred))` — map applies len to every set; sum adds them up.
- `{s for s in x if s[2] == typ}` — a set comprehension keeping one entity type.

SYSTEM DIAGRAM



From words to an entity-level score.

Segment 3 — Baseline and analysis

Gazetteer: longest match first

CODE

```
class Gazetteer:
    def __init__(self, sents, tags):
        votes = {}
        for words, t in zip(sents, tags):
            for s, e, typ in sorted(spans(t)):
                votes.setdefault(tuple(words[s:e]), Counter())[typ] += 1
        self.names = {k: c.most_common(1)[0][0] for k, c in votes.items()}
        self.longest = max(map(len, self.names))
    def tag(self, words):
        tags, i = ["O"] * len(words), 0
        while i < len(words):
            for L in range(min(self.longest, len(words) - i), 0, -1):
                typ = self.names.get(tuple(words[i:i + L]))
                if typ:
                    tags[i:i + L] = [f"B-{typ}"] + [f"I-{typ}"] * (L - 1); i += L; break
            else:
                i += 1
        return tags
```

- `votes.setdefault(tuple(words[s:e]), Counter())[typ] += 1` — a tuple of words as a dictionary key, with a Counter of the types it was given.
- `for L in range(min(self.longest, len(words) - i), 0, -1):` — a range counting down: try the longest window first.
- `for ... else:` — a for loop's else runs only if the loop did NOT break: no entry started here.

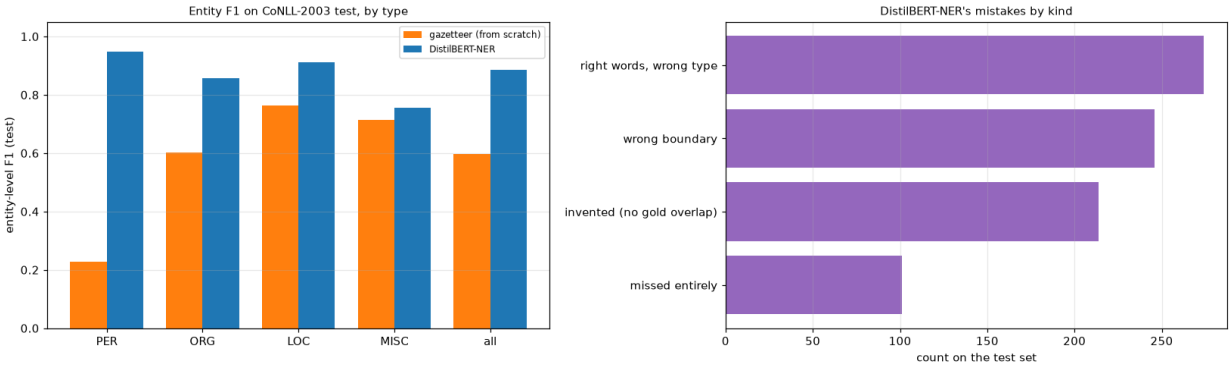
demo_failures: kinds of mistake

CODE

```
kinds = Counter()
for g, p in zip(gold, pred):
    for s in p - g:
        same = [x for x in g if (x[0], x[1]) == (s[0], s[1])]
        over = [x for x in g if x[0] < s[1] and s[0] < x[1]]
        kinds["right words, wrong type" if same else "wrong boundary" if over else "invented (no
            gold overlap)"] += 1
    for s in g:
        if not any(x[0] < s[1] and s[0] < x[1] for x in p):
            kinds["missed entirely"] += 1
```

- `p - g` — set difference: predicted spans that are not exactly right.
- `x[0] < s[1] and s[0] < x[1]` — two spans overlap if each starts before the other ends.
- `"a" if ... else "b" if ... else "c"` — a chained conditional expression picks the kind.

REAL GRAPH



Real graph from the script: F1 by type for both systems, and the model's mistakes by kind.

Quick Syntax Glossary — new or reinforced constructs

- `tok(words, is_split_into_words=True)` — tokenize text that is already split into words.
- `enc.word_ids(i)` — the word each piece came from, for batch item i.
- `set_a & set_b, set_a - set_b` — intersection and difference of sets.
- `d.setdefault(k, Counter())[v] += 1` — create a counter on first use, then count.
- `for ... else:` — the else block runs when the loop finishes without break.
- `pipeline("token-classification", aggregation_strategy="first")` — the library's NER with pieces grouped into words by their first piece.

End of Day 100 syntax reference. For the experiments and what they showed, see the Day 100 complete guide.